1. **Nguyễn Nho Trường: Thiết kế Logic Xử lý Tín hiệu Số (DSP & FIR Filtering)**
   - Viết mã logic RTL để triển khai 6 khối lọc số FIR hoạt động song song.
   - Map luồng dữ liệu trực tiếp vào 2 - 6 khối DSP/Multiplier vật lý của bo mạch Gowin OneKiwi 4K để lọc nhiễu tần số cao từ rung động cơ học của xe máy theo thời gian thực.
   - Phải đảm bảo việc triển khai không làm cạn kiệt ngân sách tài nguyên dự toán từ 2000 – 3000 LUTs.

   Vương Thành Luân: Tích hợp IP Core Toán học & Trích xuất Đặc trưng
   - Tích hợp và đồng bộ các IP Core toán học (CORDIC, MULTADDALU, MULT) để ước lượng Pitch/Roll từ dữ liệu IMU.
   - Thiết kế chuỗi logic phần cứng trích xuất các đặc trưng thống kê gồm Giá trị hiệu dụng RMS và Phương sai.
   - Triển khai bộ so sánh phần cứng: Viết logic tính tổng bình phương gia tốc ($Ax^2 + Ay^2 + Az^2$) và so sánh với ngưỡng tĩnh nhằm tạo tín hiệu đánh thức MCU thay vì tính căn bậc hai để tiết kiệm tài nguyên.
2. **Phan Hoài Quốc Minh: Quản lý Xung nhịp & Bộ đệm Dữ liệu (Clock Domains)**
   - Sử dụng IP Hard-Core PLLVR để quản lý và phân bổ tần số xung clock hệ thống (27MHz, 50MHz, 100MHz, 2.5MHz) cho toàn bộ vi mạch.
   - Khởi tạo IP Core FIFO bằng cách sử dụng 4 – 8 khối Block RAM làm bộ đệm cửa sổ trượt.
   - Mục tiêu của nhiệm vụ này là giải quyết triệt để sự chênh lệch miền xung nhịp (clock domain crossing) giữa tốc độ lấy mẫu chậm của cảm biến và khối tính toán tốc độ cao phía sau.
3. **Nguyễn Xuân Thắng: Cấu hình IP Core Giao tiếp Ngoại vi (I2C/SPI)**
   - Triển khai IP Core I2C Master để FPGA chủ động đọc dữ liệu cảm biến IMU từ MPU6050 ở mức điện áp 3.3V.
   - Cấu hình IP Core SPI Master và chân ngắt GPIO.
   - Viết logic kéo mức tín hiệu ngắt GPIO để đánh thức MCU, đồng thời tự động tạo xung clock (SCK) đẩy gói dữ liệu đặc trưng sang Sonix MCU với độ trễ micro-giây ngay khi nhận được cờ vượt ngưỡng từ khối toán học.

MÔ TẢ DỰ ÁN

1. Nguyên lý hoạt động của hệ thống Edge AI

Hệ thống hoạt động theo chuỗi xử lý nối tiếp, phân chia rõ ràng nhiệm vụ giữa phần cứng (FPGA) và phần mềm (MCU) để tối ưu hóa năng lượng và thời gian phản hồi:

- **Tiền xử lý và Đánh thức (FPGA Gowin OneKiwi 4K):** Khối FPGA giao tiếp với cảm biến MPU6050 qua chuẩn I2C để liên tục lấy mẫu gia tốc và con quay hồi chuyển. Logic RTL triển khai 6 khối lọc số FIR song song để triệt tiêu nhiễu rung động cơ học của động cơ. Thay vì sử dụng khối tính toán căn bậc hai phức tạp, FPGA tính tổng bình phương gia tốc ($Ax^2 + Ay^2 + Az^2$); nếu giá trị này vượt ngưỡng tĩnh đã định, FPGA xuất tín hiệu ngắt GPIO để đánh thức MCU khỏi trạng thái Sleep. Đồng thời, các IP Core toán học trích xuất 4 đặc trưng: Pitch, Roll, RMS, và Phương sai.
- **Truyền tải và Suy luận (MCU Sonix SN32F407):** Ngay khi đánh thức MCU, FPGA chuyển sang vai trò SPI Master, chủ động tạo xung SCK đẩy gói vector 4 đặc trưng sang MCU (SPI Slave). Vi điều khiển Sonix chạy mô hình mạng nơ-ron Autoencoder (TinyML) được viết bằng mã C thuần. Do AI chỉ được huấn luyện bằng dữ liệu chạy xe bình thường, một vụ tai nạn sẽ tạo ra dữ liệu dị thường khiến sai số tái tạo (Reconstruction Error) tăng vọt, qua đó MCU phân loại và xác nhận tai nạn.
- **Cảnh báo và Viễn thông:** Sau khi cực kì nghi ngờ đó là tai nạn, MCU điều khiển GPIO bật còi và LED đếm ngược, cấp cho người dùng một khoảng thời gian 10s để nhấn nút vật lý hủy lệnh nếu xe rơi vào trường hợp báo động giả hoặc người dùng vẫn còn "ổn" sau tai nạn. Trong 10 giây đó, MCU tiếp tục giao tiếp SPI với FPGA để đọc liên tục đặc trưng góc nghiêng (Roll) và Phương sai (Variance). Nếu Roll > 60 độ (xe đang nằm bẹp) và phương sai giảm xuống mức thấp ổn định (xe đã trượt xong, chỉ còn rung động cơ) thì sẽ "chốt" ngay mà không cần đợi. Sau đó, MCU đọc luồng UART từ module GPS u-blox NEO-M8N. Firmware dùng vòng lặp ký tự (State Machine) để bóc tách mã NMEA thay vì dùng <string.h> nhằm tránh tràn 32KB ROM. Cuối cùng, MCU gửi chuỗi AT Command qua cổng UART thứ hai đến module 4G SIM A7680C để phát tán tin nhắn SOS kèm tọa độ. Tất cả kết nối UART đều tuân thủ nguyên lý nối chéo TX vào RX và ngược lại.
- Do yêu cầu phải tận dụng càng nhiều tiện ích phần cứng nên cố nghĩ ra cái gì thêm vào để bào nát 2 cái board.

2. Thiết kế nguyên mẫu cơ khí và điện tử thực tế

Để đảm bảo hệ thống vận hành ổn định trong cốp dưới yên xe máy, nguyên mẫu yêu cầu sự tách biệt về cơ học và quy tắc đi dây điện nghiêm ngặt:

- **Cách ly cơ học (Mechanical Isolation):** Hệ thống được chia làm hai khối độc lập bên trong hộp chứa. Cảm biến MPU6050 bắt buộc phải được bắt vít cố định cứng vào hộp chứa gắn ở cốp xe nhằm hạn chế rung và chuyển động tương đối giữa thiết bị và xe, giúp dữ liệu động học không bị sai lệch. Khối bo mạch PCB chính (chứa MCU, FPGA, module 4G, GPS, anten 38dBi, pin Li-ion) có khối lượng lớn nên phải được bọc trong lớp foam giảm sốc nhằm triệt tiêu lực G-force, bảo vệ khe cắm SIM và các mối hàn không bị đứt gãy do mỏi kim loại.
- **Phân phối công suất (Power Distribution):** Thiết bị hoạt động bằng viên pin Li-ion độc lập hoàn toàn, được quản lý sạc và bảo vệ quá xả bởi module IP5306. Chân cấp nguồn của module 4G LTE SIM A7680C yêu cầu dải điện áp từ 3.7-4.2V, do đó nhánh này bắt buộc phải được hàn trực tiếp vào ngõ ra của mạch Ip5306 (nối song song với cell pin) để viên pin gánh trực tiếp các xung dòng tải đỉnh khi phát sóng RF, giúp chống sụt áp toàn hệ thống. Nhánh điện áp còn lại đi qua mạch kích áp MT3608 để nâng lên mức 5V tĩnh, cấp thẳng vào chân VIN của bo mạch Gowin OneKiwi 4K và Sonix SN32F407 EVK. Cuối cùng, ta trích xuất nguồn 3.3V từ mạch hạ áp (LDO) đã được tích hợp sẵn trên chính các bo mạch này để nuôi cảm biến IMU MPU6050 và module định vị GPS u-blox NEO-M8N.
- **Toàn vẹn tín hiệu (Signal Integrity):** Cáp tín hiệu I2C nối giữa khối PCB chính và cụm IMU phải sử dụng dây lõi mềm nhiều sợi (stranded wire) để chịu được chu kỳ uốn cong liên tục. Bắt buộc hàn thêm điện trở kéo lên (pull-up) 2.2kΩ tại các đường tín hiệu SDA và SCL ngay sát ngõ vào FPGA để sạc nhanh điện dung ký sinh trên đường dây, đảm bảo sườn xung logic giữ đúng mức điện áp 3.3V trong môi trường nhiễu điện từ khắc nghiệt.

Mua:
- khay pin song song 4 viên loại nối dây, *MT3608*, *TP4056 (việt nic)*, trụ đồng nâng mạch (Việt nic), *Module 4G LTE SIM A7680C* (kèm Anten), *Module u-blox NEO-M6N* (kèm Anten 38dBi), tụ hóa, hộp nhựa abs có nắp (Nguyễn Hiền), khóa (mua sau), đế (làm sau), trụ cao su giảm chấn ren M3 1 đầu hoặc 2 đầu, xốp giảm sốc.
-2 hộp nhựa lồng nhau, hộp nhựa ABS có nắp nằm trong, đến lớp giảm sốc rồi đựng trong hộp nhựa lớn cố định với cốp xe.
Test: Khả năng lọc nhiễu + Phát hiện tai nạn và cảnh báo

Để 3 thành viên cùng lập trình cho một vi điều khiển **Sonix SN32F407** chỉ có **32KB Flash ROM và 8KB SRAM** mà không dẫm chân lên nhau, trước hết chúng ta phải nhìn thẳng vào bản chất vật lý của con chip này:

1. **Nguyên nhân 1 (Xung đột phần cứng):** Toàn bộ ngoại vi của SN32F407 (SPI0, UART0, UART1, GPIO, WDT) đều dùng chung các thanh ghi cấp xung nhịp (SYS1_AHBCLKEN) và định tuyến chân (PFPA). Nếu 3 người tự ý gán lệnh = thay vì |= vào cùng một thanh ghi, code của người sau sẽ xóa sạch cấu hình ngoại vi của người trước.
2. **Nguyên nhân 2 (Chết thời gian thực do không có RTOS):** Vì lập trình Bare-metal (vòng lặp while(1) đơn luồng), nếu người làm GPS dùng vòng lặp chờ chuỗi NMEA quá lâu mà không trả quyền điều khiển hoặc không nạp lại Watchdog (WDT_FEED), chip sẽ tưởng hệ thống bị treo và tự động Reset toàn mạch.
3. **Hệ quả tất yếu:** Bắt buộc phải tách chương trình theo **3 tầng độc lập dọc theo luồng đi của dữ liệu vật lý** (Đầu vào SPI/Nguồn → Xử lý AI & Điều khiển → Đầu ra GPS/4G), và khóa chặt giao tiếp giữa 3 người bằng **1 file Header hợp đồng dùng chung (system_shared.h)**.

SƠ ĐỒ PHÂN CHIA NHIỆM VỤ CHO 3 THÀNH VIÊN

Thành viên	Phụ trách Phân hệ	Các File Bàn giao	Ngoại vi & Thanh ghi Độc quyền Quản lý
Thành viên 1 *(Kỹ sư Hệ thống, Nguồn &*	**Tầng Đánh thức & Nhận đặc trưng:** Quản lý xung nhịp, chống sụt áp LVD, chống treo WDT, chế độ ngủ Sleep Mode,	system_shared.h mcu_sys_spi.h	SYS0, SYS1, PMU, WDT, SYSTICK, SPI0 (Cụm J12: P1.0, P1.1, P1.2, P1.8), Ngắt P2.8 (Cụm J7).

Giao tiếp FPGA)	ngắt đánh thức P2.8 và đọc gói 64-bit từ FPGA qua SPI0 Slave.	mcu_sys_spi.c	
Thành viên 2 *(Kỹ sư Thuật toán TinyML & Giao diện Cảnh báo)*	**Tầng Quyết định & Tương tác:** Suy luận mạng nơ-ron Autoencoder (Q8.8), quét nút nhấn hủy SOS (SW3), nút bơm lỗi kiểm thử (SW4..SW6), điều khiển còi PZ1 đếm ngược 10s và ghép nối main.c.	tinyml_fsm.h tinyml_fsm.c main.c	Trọng số Flash ROM (const), Còi PZ1 (P3.0), Nút nhấn SW3..SW6 (P2.4..P2.7), Đèn D8 (P2.10), D9 (P1.5).
Thành viên 3 *(Kỹ sư Định vị GPS & Viễn thông*	**Tầng Cứu hộ & Truyền thông:** Máy trạng thái bóc tách tọa độ NMEA từ GPS qua UART0, tập lệnh AT gửi SMS/Gọi điện qua	uart_gps_4g.h uart_gps_4g.c chỗ này của tao mà(?)	UART0 (Cụm J10: P3.1, P3.2), UART1 (Cụm J7: P2.2, P2.3), Chân Reset 4G (P2.1), Đèn D10 (P1.6), D11 (P1.7).

4G LTE)	UART1 (4G), chân kích Reset cứng 4G (P2.1) và hàm xuất CSV thu thập dữ liệu train AI.		

HỢP ĐỒNG GIAO TIẾP CHUNG: system_shared.h

(Cả 3 thành viên cùng #include "system_shared.h". Không ai được tự ý sửa địa chỉ thanh ghi trong file này).

C

```
#ifndef SYSTEM_SHARED_H
#define SYSTEM_SHARED_H

#include <stdint.h>

/*
================================================================
============
 * 1. ÁNH XẠ ĐỊA CHỈ THANH GHI VẬT LÝ SONIX SN32F407 (THEO MEMORY
MAP)
 *
================================================================
============ */
// Quản lý Hệ thống, Nguồn & Xung nhịp (SYS0: 0x40060000, SYS1:
0x4005E000, PMU: 0x40032000)
#define SYS0_ANBCTRL    (*(volatile uint32_t *)(0x40060000UL + 0x00UL)) //
Điều khiển dao động nội IHRC[cite: 53, 75]
#define SYS0_RSTST      (*(volatile uint32_t *)(0x40060000UL + 0x14UL)) //
Trạng thái nguồn gây Reset[cite: 53, 77]
#define SYS0_LVDCTRL    (*(volatile uint32_t *)(0x40060000UL + 0x18UL)) //
Giám sát điện áp thấp LVD[cite: 53, 78]
#define SYS1_AHBCLKEN   (*(volatile uint32_t *)(0x4005E000UL + 0x00UL)) //
Cấp xung nhịp AHB cho ngoại vi[cite: 53, 81]
```

#define SYS1_APBCP1   (*(volatile uint32_t *)(0x4005E000UL + 0x08UL)) // Bộ chia xung nhịp cho WDT[cite: 53, 82]
#define PMU_CTRL   (*(volatile uint32_t *)(0x40032000UL + 0x40UL)) // Điều khiển chế độ Sleep/Deep-sleep[cite: 53, 90]

// Định tuyến chân ngoại vi PFPA (Base: 0x40042000)
#define PFPA_UART0   (*(volatile uint32_t *)(0x40042000UL + 0x20UL)) // Chọn chân cho UART0 (GPS)[cite: 53, 99]
#define PFPA_UART1   (*(volatile uint32_t *)(0x40042000UL + 0x24UL)) // Chọn chân cho UART1 (4G)[cite: 53, 99]
#define PFPA_SPI0   (*(volatile uint32_t *)(0x40042000UL + 0x50UL)) // Chọn chân cho SPI0 (FPGA)[cite: 53, 100]

// Thanh ghi GPIO Port 1 (0x40046000), Port 2 (0x40048000), Port 3 (0x4004A000)
#define GPIO1_DATA   (*(volatile uint32_t *)(0x40046000UL + 0x00UL)) // Dữ liệu vào/ra Port 1[cite: 53, 92]
#define GPIO1_MODE   (*(volatile uint32_t *)(0x40046000UL + 0x04UL)) // Hướng chân Port 1 (0:In, 1:Out)[cite: 92]
#define GPIO1_CFG   (*(volatile uint32_t *)(0x40046000UL + 0x08UL)) // Cấu hình trở kéo Port 1[cite: 92]
#define GPIO1_BSET   (*(volatile uint32_t *)(0x40046000UL + 0x24UL)) // Bật bit nguyên tử Port 1[cite: 95]
#define GPIO1_BCLR   (*(volatile uint32_t *)(0x40046000UL + 0x28UL)) // Xóa bit nguyên tử Port 1[cite: 95]

#define GPIO2_DATA   (*(volatile uint32_t *)(0x40048000UL + 0x00UL)) // Dữ liệu vào/ra Port 2[cite: 53, 92]
#define GPIO2_MODE   (*(volatile uint32_t *)(0x40048000UL + 0x04UL)) // Hướng chân Port 2 (0:In, 1:Out)[cite: 92]
#define GPIO2_CFG   (*(volatile uint32_t *)(0x40048000UL + 0x08UL)) // Cấu hình trở kéo Port 2[cite: 92]
#define GPIO2_IS   (*(volatile uint32_t *)(0x40048000UL + 0x0CUL)) // Ngắt theo sườn (0) hay mức (1)[cite: 93]
#define GPIO2_IBS   (*(volatile uint32_t *)(0x40048000UL + 0x10UL)) // Ngắt 1 sườn (0) hay 2 sườn (1)[cite: 94]

```
#define GPIO2_IEV       (*(volatile uint32_t *)(0x40048000UL + 0x14UL)) // Sườn lên (0) hay sườn xuống (1)[cite: 94]
#define GPIO2_IE        (*(volatile uint32_t *)(0x40048000UL + 0x18UL)) // Cho phép ngắt trên Port 2[cite: 94]
#define GPIO2_RIS       (*(volatile uint32_t *)(0x40048000UL + 0x1CUL)) // Cờ báo ngắt thô Port 2[cite: 94]
#define GPIO2_IC        (*(volatile uint32_t *)(0x40048000UL + 0x20UL)) // Xóa cờ ngắt Port 2[cite: 95]
#define GPIO2_BSET      (*(volatile uint32_t *)(0x40048000UL + 0x24UL)) // Bật bit nguyên tử Port 2[cite: 95]
#define GPIO2_BCLR      (*(volatile uint32_t *)(0x40048000UL + 0x28UL)) // Xóa bit nguyên tử Port 2[cite: 95]

#define GPIO3_MODE      (*(volatile uint32_t *)(0x4004A000UL + 0x04UL)) // Hướng chân Port 3[cite: 53, 92]
#define GPIO3_BSET      (*(volatile uint32_t *)(0x4004A000UL + 0x24UL)) // Bật bit nguyên tử Port 3[cite: 95]
#define GPIO3_BCLR      (*(volatile uint32_t *)(0x4004A000UL + 0x28UL)) // Xóa bit nguyên tử Port 3[cite: 95]

// Ngoại vi SPI0 (Base: 0x4001C000)
#define SPI0_CTRL0      (*(volatile uint32_t *)(0x4001C000UL + 0x00UL)) // Thanh ghi điều khiển SPI0 0[cite: 30, 53]
#define SPI0_CTRL1      (*(volatile uint32_t *)(0x4001C000UL + 0x04UL)) // Thanh ghi điều khiển pha/cực SPI0[cite: 30, 181]
#define SPI0_STAT       (*(volatile uint32_t *)(0x4001C000UL + 0x0CUL)) // Thanh ghi trạng thái FIFO SPI0[cite: 30]
#define SPI0_DATA       (*(volatile uint32_t *)(0x4001C000UL + 0x18UL)) // Thanh ghi dữ liệu 16-bit SPI0[cite: 30]

// Ngoại vi UART0 - GPS (0x40016000) & UART1 - 4G (0x40056000)
#define UART0_RB        (*(volatile uint32_t *)(0x40016000UL + 0x00UL)) // Bộ đệm nhận UART0[cite: 32, 53]
#define UART0_DLL       (*(volatile uint32_t *)(0x40016000UL + 0x00UL)) // Byte thấp chia Baud UART0
#define UART0_DLM       (*(volatile uint32_t *)(0x40016000UL + 0x04UL)) // Byte
```

cao chia Baud UART0[cite: 32]
#define UART0_FIFOCTRL (*(volatile uint32_t *)(0x40016000UL + 0x08UL)) // Điều khiển FIFO UART0[cite: 32]
#define UART0_LC   (*(volatile uint32_t *)(0x40016000UL + 0x0CUL)) // Định dạng khung truyền UART0[cite: 32]
#define UART0_LS   (*(volatile uint32_t *)(0x40016000UL + 0x14UL)) // Trạng thái đường truyền UART0[cite: 32]
#define UART0_FD   (*(volatile uint32_t *)(0x40016000UL + 0x28UL)) // Chia Baud phân số UART0[cite: 32]
#define UART0_CTRL   (*(volatile uint32_t *)(0x40016000UL + 0x30UL)) // Kích hoạt TX/RX UART0[cite: 32]

#define UART1_RB   (*(volatile uint32_t *)(0x40056000UL + 0x00UL)) // Bộ đệm nhận UART1[cite: 32, 53]
#define UART1_TH   (*(volatile uint32_t *)(0x40056000UL + 0x00UL)) // Bộ đệm phát UART1[cite: 32]
#define UART1_DLL   (*(volatile uint32_t *)(0x40056000UL + 0x00UL)) // Byte thấp chia Baud UART1[cite: 32]
#define UART1_DLM   (*(volatile uint32_t *)(0x40056000UL + 0x04UL)) // Byte cao chia Baud UART1[cite: 32]
#define UART1_FIFOCTRL (*(volatile uint32_t *)(0x40056000UL + 0x08UL)) // Điều khiển FIFO UART1[cite: 32]
#define UART1_LC   (*(volatile uint32_t *)(0x40056000UL + 0x0CUL)) // Định dạng khung truyền UART1[cite: 32]
#define UART1_LS   (*(volatile uint32_t *)(0x40056000UL + 0x14UL)) // Trạng thái đường truyền UART1[cite: 32]
#define UART1_FD   (*(volatile uint32_t *)(0x40056000UL + 0x28UL)) // Chia Baud phân số UART1[cite: 32]
#define UART1_CTRL   (*(volatile uint32_t *)(0x40056000UL + 0x30UL)) // Kích hoạt TX/RX UART1[cite: 32]

// Watchdog Timer (0x40010000), SysTick & NVIC
#define WDT_CFG   (*(volatile uint32_t *)(0x40010000UL + 0x00UL)) // Cấu hình Watchdog[cite: 53, 174]
#define WDT_TC   (*(volatile uint32_t *)(0x40010000UL + 0x08UL)) // Hằng số thời gian Watchdog[cite: 174]

```
#define WDT_FEED        (*(volatile uint32_t *)(0x40010000UL + 0x0CUL)) // Nạp lại Watchdog[cite: 175]
#define SYSTICK_CTRL    (*(volatile uint32_t *)(0xE000E010UL))          // Điều khiển SysTick[cite: 55]
#define SYSTICK_LOAD    (*(volatile uint32_t *)(0xE000E014UL))          // Giá trị nạp lại SysTick[cite: 55]
#define SYSTICK_VAL     (*(volatile uint32_t *)(0xE000E018UL))          // Giá trị đếm hiện tại SysTick[cite: 56]
#define NVIC_ISER       (*(volatile uint32_t *)(0xE000E100UL))          // Bật ngắt trong NVIC[cite: 58]

/*
============================================================
 * 2. MACRO ĐIỀU KHIỂN ĐÈN LED CHẨN ĐOÁN (ACTIVE-LOW) & CÒI BUZZER TRÊN EVK
 *
============================================================ */
#define LED_D8_SPI_ON()   (GPIO2_BCLR = (1UL << 10)) // D8 (P2.10): Sáng khi nhận gói SPI từ FPGA[cite: 95, 191]
#define LED_D8_SPI_OFF()  (GPIO2_BSET = (1UL << 10))
#define LED_D9_AI_ON()    (GPIO1_BCLR = (1UL << 5))  // D9 (P1.5):  Nháy khi AI xác nhận tai nạn[cite: 95, 191]
#define LED_D9_AI_OFF()   (GPIO1_BSET = (1UL << 5))
#define LED_D10_GPS_ON()  (GPIO1_BCLR = (1UL << 6))  // D10 (P1.6): Sáng khi GPS đã khóa vệ tinh[cite: 95, 191]
#define LED_D10_GPS_OFF() (GPIO1_BSET = (1UL << 6))
#define LED_D11_4G_ON()   (GPIO1_BCLR = (1UL << 7))  // D11 (P1.7): Sáng khi 4G phản hồi AT OK[cite: 95, 191]
#define LED_D11_4G_OFF()  (GPIO1_BSET = (1UL << 7))

#define BUZZER_ON()       (GPIO3_BSET = (1UL << 0))  // Còi PZ1 (P3.0)[cite: 95, 191]
#define BUZZER_OFF()      (GPIO3_BCLR = (1UL << 0))
```

```
/*
============================================================
============
 * 3. BIẾN TOÀN CỤC CHIA SẺ GIỮA 3 THÀNH VIÊN
 *
============================================================
============ */
extern volatile int16_t g_features[4];      // [0]=Pitch, [1]=Roll, [2]=RMS^2,
[3]=Variance
extern volatile uint8_t g_spi_pkt_ready;    // Set = 1 bởi Thành viên 1, xóa = 0 bởi
Thành viên 2
extern char             g_gps_coord[32];    // Chuỗi tọa độ NMEA cập nhật bởi
Thành viên 3

#endif /* SYSTEM_SHARED_H */
```

Mua module thẻ nhớ=>Hàn đế pin=> Lập trình và kết nối lên FPGA để lấy dữ liệu=> Đem đầu đọc thẻ USB để lấy dữ liệu bỏ vào máy tính => Sau đó train AI=> Tích hợp lên board MCU.

Lấy dữ liệu

-Nối chân dùng 4 chân: CS-46, SCK-43, MOSI-42, MISO-41. Nguồn 5V. Buộc phải hàn dây đực cái vào trước. ===> Dùng để đọc IMU lưu vào thẻ nhớ.

-Thêm code cho module đọc thẻ.

-Phải có đầu đọc thẻ USB để lấy dữ liệu mà train AI .

Chế tạo:

-Khoan lỗ, nối dây, chế hoàn toàn hộp

-Mua chống sốc, keo, ốc vít.

Mua

-Module đọc thẻ, sim, dây cắm, mũi khoan M3, ốc M3.

-5 chân GPIO của FPGA: đỏ-nâu-xanh biển-xanh lá-cam = 46 44 43 42 41

-2 chân 4g: rx đen, tx tím